

HTTP API на Go

Лекция и практика / 90 минут



Структура курса

01.  Основа backend-сервиса
02.  Бизнес логика
03.  Взаимодействие сервисов
04.  Дополнительные темы: CI/CD и AI



Правила взаимодействия

1. Активное участие на занятиях — дополнительные баллы
2. Камеры на онлайн-занятиях — обязательно включены
3. Использование ИИ не запрещено — но нужно понимать, что, как и почему сделано: любой сданный код или текст нужно уметь объяснить
4. Вопросы по дисциплине к преподавателям — в общий чат, точечные личные вопросы — куратору или методисту

Структура проекта



Зачем нужна структура



Рост проекта и поддержка

Новому разработчику проще разобраться в коде.



Единообразие

Понятно, где точки входа, бизнес-логика и общие пакеты.

Репозиторий golang-standards

```
project/
├── cmd/
│   └── app/main.go
├── internal/
│   ├── config/
│   └── database/
├── pkg/
│   └── mypackage/
└── api/
```

Пример структуры

Это название репозитория.
Он не является
официальным
стандартом Go.

Выбирайте каталоги,
которые нужны проекту.

github.com/golang-standards/project-layout

Основные каталоги

1

cmd

Точки входа:
приложение и команды.

Здесь размещают
пакеты main.

2

internal

Внутренние пакеты.

Импорт ограничен
деревом
родительского каталога.

3

pkg

Переиспользуемые
пакеты.

Необязательная
конвенция.
Публичный пакет может
жить и в корне.

Структура и архитектура

	Структура	Архитектура
Что это	Организация файлов, пакетов и модулей	Взаимодействия и зависимости компонентов
Отвечает на	«Где это лежит?»	«Как это работает?»
Пример	cmd/, internal/, pkg/, handlers/	Слои, домены, use case, clean architecture

Сколько точек входа (пакетов main) в репозитории Kubernetes?

1 1

2 5

3 10

4 >10

github.com/kubernetes/kubernetes

Пакет locker в Docker Compose - это...

1 Пакет с тестами

2 Пакет с main

3 Внутренний пакет

4 Пакет, который можно
переиспользовать

github.com/docker/compose

Сервер и хэндлеры



Пакет HTTP

В стандартной библиотеке Go

01 Веб-сервер

`http.Server`, `ListenAndServe`

02 Маршрутизация

`http.ServeMux`

03 HTTP-клиент

`http.Client`

04 Формы и JSON

`net/http` + `encoding/json`

`net/http`



[Документация net/http](#)

Минимальный работающий сервер

```
func main() {  
    mux := http.NewServeMux()  
    mux.HandleFunc("GET /ping", ping)  
  
    log.Println("listening on :8080")  
    http.ListenAndServe(":8080", mux)  
}
```

Запуск

`go run ./cmd/server`

Проверка:

`curl -i localhost:8080/ping`

Ответ:

`200 OK`

http.Handler и http.HandlerFunc

```
type Handler interface {  
    ServeHTTP(  
        ResponseWriter,  
        *Request,  
    )  
}  
  
func ping(w http.ResponseWriter, r *http.Request){  
    // формируем ответ  
}
```

Один контракт

Request содержит
запрос.

ResponseWriter
позволяет
записать ответ.

http.HandlerFunc(ping)
адаптирует функцию
к интерфейсу Handler.

Маршруты в стандартном ServeMux

```
mux.HandleFunc(  
    "GET /items/{id}", getItem,  
)  
  
func getItem(w http.ResponseWriter, r *http.Request){  
    rawID := r.PathValue("id")  
    id, err := strconv.Atoi(rawID)  
  
    if err != nil || id <= 0 {  
        // вернуть 400 и выйти  
    }  
}
```

Для этих возможностей сторонний роутер не требуется.

Go 1.22+

Метод входит в паттерн.

{id} обозначает один сегмент пути.

PathValue возвращает строку: число проверяем в обработчике.

Порядок записи HTTP-ответа

1

Заголовки

Content-Type:
application/json

2

Статус

WriteHeader(201)

3

Тело

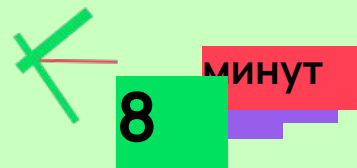
Encoder.Encode(item)

Первая запись тела без WriteHeader отправляет статус 200.

Практика 1. GET /items/{id}

В заготовке уже работает /ping. Добавим JSON-ответ для объявления.

- 1 Допишите обработчик GET /items/{id}.
- 2 Проверьте id: положительное целое число.
- 3 Верните {"id":42,"title":"lamp"} для id=42.
- 4 Проверьте 200, неверный id: 400, другой id: 404.



**Готово, если
статус и JSON
совпадают
с запросом**

Разбор первой ручки

```
id, err := strconv.Atoi(r.PathValue("id"))
if err != nil || id <= 0 {
    w.WriteHeader(http.StatusBadRequest)
    return
}
if id != 42 {
    w.WriteHeader(http.StatusNotFound)
    return
}
w.Header().Set("Content-Type", "application/json")
if err := json.NewEncoder(w).Encode(
    Item{42, "lamp"},
); err != nil {
    log.Print(err)
}
```

Проверяем
границы

/items/42
200 + JSON

/items/abc
400, без тела

/items/99
404, без тела

Middleware и жизненный цикл

03

Стандартный mux или chi

В обоих: методы, параметры пути и http.Handler

Задача	net/http.ServeMux	chi
Вложенные маршруты	Handle + подроутер	Route, Group
Middleware	Оборачиваем handler	Use или обёртка
	Встроен в Go	Отдельная библиотека

Выбор зависит от нужных возможностей и правил команды.

Порядок выполнения middleware

1

Logging: вход

Запоминаем
время начала

2

Handler

Выполняем операцию
и пишем ответ

3

Logging: выход

Считаем длительность
и записываем лог

Logging(Auth(handler)): на входе Logging, затем Auth; на выходе - обратно.

Логирование длительности

```
func Logging(h http.Handler) http.Handler {  
  
    return http.HandlerFunc(func(  
        w http.ResponseWriter,  
        r *http.Request,  
    ){  
        started := time.Now()  
  
        h.ServeHTTP(w, r)  
  
        log.Printf("%s %s took %s",  
            r.Method, r.URL.Path,  
            time.Since(started),  
        )  
    })  
  
}
```

Подключение

handler := Logging(mux)

Передаём handler
в http.Server.

Обёртка одна
для всех маршрутов.

Graceful Shutdown

03

Корректная остановка сервера

1

Сигнал

SIGTERM
или Ctrl+C

2

Shutdown

Закрываем
listener

3

Ожидание

Завершаем
активные
запросы

4

Выход

Закрываем
ресурсы

Задаём отдельный таймаут остановки. `Server.Close` закрывает соединения сразу.

Сигнал и отдельный контекст остановки

```
ctx, stop := signal.NotifyContext(
    context.Background(),
    os.Interrupt, syscall.SIGTERM,
)

defer stop()

<-ctx.Done()

shutdownCtx, cancel := context.WithTimeout(
    context.Background(), 10*time.Second,
)
defer cancel()

err := srv.Shutdown(shutdownCtx)

if err != nil {
    log.Printf("shutdown: %v", err)
}
```

При остановке

ListenAndServe работает
в отдельной горутине.

http.ErrServerClosed
при остановке ожидаем.

Контекст shutdown
создаём заново,
он ещё не отменён.

API-FIRST

04

API-first: контракт до реализации

1

Контракт

Пути, поля, статусы
и ошибки в OpenAPI

2

Генерация

Go-типы
и интерфейс сервера

3

Реализация

Логика обработки
и тесты поведения

Клиент и сервер используют общий контракт. Генерация не заменяет тестирование.

OpenAPI: операция и параметры

```
paths:
  /items/{id}:
    get:
      operationId: getItem
      parameters:
        - name: id
          in: path
          required: true
          schema: {type: integer, minimum: 1}
      responses:
        "200":
          description: Item found
```

Это фрагмент. Полный openapi.yaml находится в материалах практики.

Что описали

paths - адрес операции

operationId - имя
для генератора

parameters - параметры
и их ограничения

responses - варианты
ответа

OpenAPI: модель ответа

```
components:
  schemas:
    Item:
      type: object
      required: [id, title]
      properties:
        id:
          type: integer
        title:
          type: string
```

```
# В схеме ответа:
$ref: "#/components/schemas/Item"
```

Одна модель

required задаёт
обязательные поля.

\$ref позволяет
переиспользовать Item
в разных операциях.

Из модели получим
Go-структуру.

Генерация интерфейса сервера

```
// Сгенерированный интерфейс
type ServerInterface interface {
    GetItem(w http.ResponseWriter,
            r *http.Request, id int)
}

// Регистрируем свою реализацию
r := chi.NewRouter()
api.HandlerFromMux(server, r)
```

Валидация тела и бизнес-правила требуют отдельной реализации.

Команда

make generate

Генерируем типы
и chi-server.

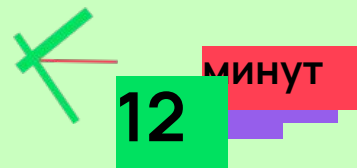
Свою логику пишем
в реализации
интерфейса.

gen.go руками не меняем.

Практика 2. Контракт GET /orders/{id}

Опишем получение заказа и получим Go-интерфейс из OpenAPI.

- 1 В api/openapi.yaml добавьте GET /orders/{id}.
- 2 Query-параметр status (string).
Ответ: 200 (успех), 400 (status пустой).
- 3 Запустите
make api-generate-get-orders
- 4 Реализуйте метод. Проверьте успех и ошибку ввода.



**Готово, если
GET даёт 200,
пустой status
даёт 400**

Проверка GET /orders/{id}

```
# status заполнен
curl -i \
  "http://localhost:8080/orders/42?status=active"
# Ожидаем: 200 OK

# status пустой
curl -i \
  "http://localhost:8080/orders/42?status="
# Ожидаем: 400 Bad Request
```

Что проверяем

Параметр status передаём в URL.

Эти GET-запросы отправляем без тела.

Проверяем HTTP-статус и тело ответа.

Контракт HTTP API

02

Идемпотентность и повтор запроса

Отправляем один и тот же запрос дважды

Вызов

PUT /items/42

POST /items

Первый

№42: «lamp»

Создано №43

Повтор

№42: «lamp»

Создано №44

Состояние то же

Возможен дубликат

Идемпотентность описывает эффект на сервере. Статус и тело ответа могут отличаться.

Ресурсы и HTTP-методы

Операция	Метод и путь	Типичный успех
Список	GET /items	200
Одно объявление	GET /items/{id}	200
Создать	POST /items	201
Полностью заменить	PUT /items/{id}	200 / 204
Частично изменить	PATCH /items/{id}	200 / 204
Удалить	DELETE /items/{id}	204

REST шире этой таблицы. Здесь - соглашения для нашего HTTP API.

Что теперь можно сделать

1

Поднять HTTP API

ServeMux выбирает обработчик.
Handler читает запрос и записывает ответ.

2

Описать контракт

Методы, статусы и JSON согласованы в OpenAPI.
Go-типы генерируются.

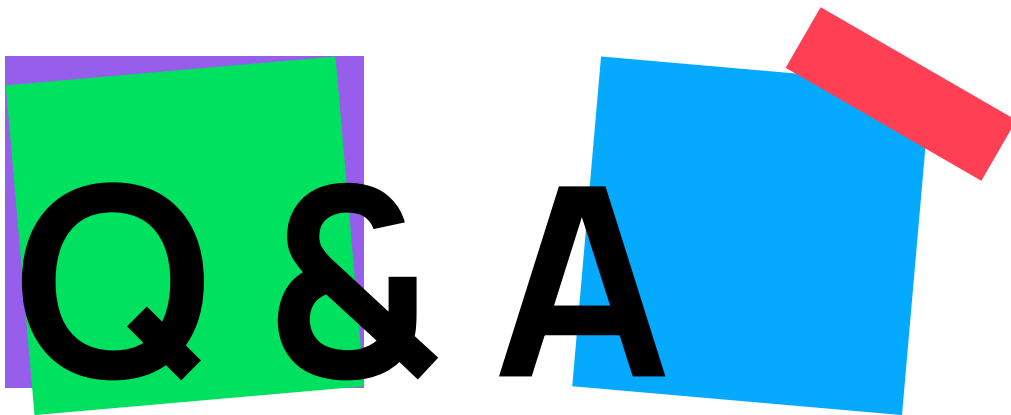
3

Проверить поведение

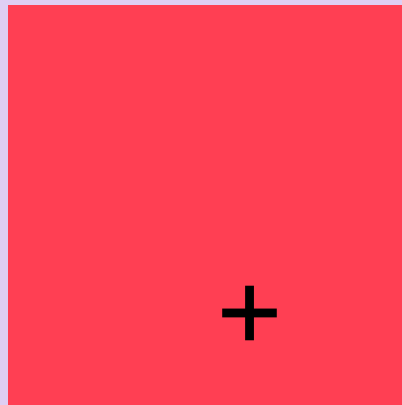
curl показывает ответ.
Middleware пишет лог.
Shutdown завершает работу.

Дальше - хранилище данных и развитие структуры сервиса.

Вопросы



Дополнительные примеры



Документация и примеры

Материалы для повторения

go.dev/doc/modules/layout - организация Go-модуля

pkg.go.dev/net/http - Handler, Server и Shutdown

go.dev/blog/routing-enhancements - маршруты Go 1.22

github.com/oapi-codegen/oapi-codegen - генерация

github.com/go-chi/chi - роутер и middleware